

5.类型转换

隐式转换：

```
int a = 5;  
double b = a;    // int → double, 自动转换
```

显式转换：

- 明确指定类型进行转换

`(<TYPE>) var`

- 将var的值转换为TYPE类型的结果
- `(int) 3.14` -- 将3.14转换为int类型，即3
- 是一个一元运算符

转换逻辑：

- 整数 → 浮点数
 - 将整数用对应的浮点数表示，计算指数
- 浮点数 → 整数
 - 截断，不四舍五入取整数部分
- 不同大小的类型
 - 截断或补齐长度（视符号补0或1）

算数转换：

- 当两个不同类型的操作数进行运算时，统一为“更高级”类型
- 转换顺序 从高到低

- `long double ← double ← float ←`
- `unsigned long long ← long long ←`
- `unsigned long ← long ←`
- `unsigned int ← int`

数值损失与溢出:

- 整数在向更小的类型转换时, 可能出现数据溢出

- `int32_t i32 = (uint8_t)300;`

- 此时数据溢出为44

- 转换为浮点数时, 可能出现精度损失

- `float f = (float)d; -- (d: double)`

- 可能损失d后面的小数位

- 当浮点数的值超出目标浮点类型的表示范围时, 会发生溢出

- 通常为 `double → float`

- C标准规定:

- 上溢: 结果过大, 通常导致无穷大 ($\pm\text{inf}$)

- 下溢: 结果过小 (接近零), 通常导致为零或次正规数

拓展:

- 结构体在内存中是连续存储的, 如

- `struct Point { int x; int y }` 在内存中存储为:

int 4byte (x)	int 4byte (y)
---------------	---------------

- 指向 `Point p` 的指针 `&p` 则是指向 `Point` 的首个元素, 即

int 4byte (x)	int 4byte (y)
&p	

- 指向 `Point p` 的指针 `&p` 则是指向 `Point` 的首个元素，即

int 4byte (x)	int 4byte (y)
<code>&p</code>	

- 假设有 `struct ColorPoint {struct Point p; int color;}`
- 则将 `ColorPoint *cp` 直接转换为 `Point*` 是否是安全的？
 - 即 `Point *p = (Point*)cp;`
- 进一步的，如果用 `ColorPoint` 作为 `Point` 调用函数，是否符合预期？
- 反之如何？这么做有什么优势？